Practical-7

A computer system has four different types of resources (printer, ROM, hard
disk and RAM) and it needs to complete execution of five processes (Google
Drive, Firefox, Word Processor, Excel and PowerPoint)' The number of allocated resources, maximum needed resources to complete the execution

```
  GNU nano 7.2                                   banker.c *
#include <stdio.h>

int main()
{
    int i, j, k;
    int processes = 5;
    int resources = 4;

    int allocation[5][4] = {
        {0,0,1,4},
        {0,6,3,2},
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4}
    };

    int max[5][4] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[4] = {1,6,2,0};

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
GNU nano 7.2                                                    banker.c *
    {
        int found = 0;

        for(i=0;i<processes;i++)
        {
            if(finish[i] == 0)
            {
                int flag = 0;

                for(j=0;j<resources;j++)
                {
                    if(need[i][j] > available[j])
                    {
                        flag = 1;
                        break;
                    }
                }

                if(flag == 0)
                {
                    safeSeq[count++] = i;

                    for(k=0;k<resources;k++)
                        available[k] += allocation[i][k];

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
GNU nano 7.2                                                    banker.c *

    int available[4] = {1,6,2,0};

    int need[5][4];
    int finish[5] = {0};
    int safeSeq[5];
    int count = 0;

    printf("\nNEED MATRIX:\n");

    for(i=0;i<processes;i++)
    {
        for(j=0;j<resources;j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }

    while(count < processes)
    {
        int found = 0;

        for(i=0;i<processes;i++)

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
GNU nano 7.2                                                    banker.c *
                        available[k] += allocation[i][k];

                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0)
        {
            printf("\nSystem is NOT in safe state\n");
            return 0;
        }
    }

    printf("\nSystem is in SAFE state\nSafe sequence is:\n");

    for(i=0;i<processes;i++)
        printf("P%d ", safeSeq[i]);

    printf("\n");

    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```